Gotta Classify ‘Em All

Gruppo di progetto

- Marika Ferrara, 776016, m.ferrara61@studenti.uniba.it

Link progetto Github: https://github.com/markk03/Progetto_ICon24-25.git

AA 2024-2025

1. Introduzione
- Sommario
- Elenco argomenti di interesse

2. Rappresentazione della conoscenza e ragionamento
- Sommario
- Strumenti utilizzati
- Decisioni di progetto
- Valutazione

3. Apprendimento supervisionato
- Sommario
- Strumenti utilizzati
- Decisioni di progetto
- Valutazione
  - Risultati della fase baseline
  - Risultati della fase OntoBK

4. Risoluzione di vincoli
- Sommario
- Strumenti utilizzati
- Decisioni di progetto
- Valutazione

Conclusioni

Sviluppi futuri

Riferimenti bibliografici

1. Introduzione

Il dominio di interesse di questo progetto è l’analisi e la composizione di squadre competitive nel panorama dei Pokémon, basandosi sui Tier (fasce di utilizzo) definiti dalla community Smogon, la più grande comunità online dedicata al battling competitivo di Pokémon. Questo ecosistema è stato scelto per la sua longevità e per la sua ricchezza in termini di elementi semantici e dinamiche tattiche.

L’obiettivo centrale è mostrare come l’integrazione di queste componenti produca un sistema di supporto alle decisioni nella costruzione di team bilanciati molto più evoluto rispetto ad un approccio puramente statistico.

Sommario

Il progetto mira a dimostrare l’efficacia di un sistema ibrido che fonde apprendimento supervisionato, ragionamento automatico e risoluzione di vincoli.

L’architettura si sviluppa su tre fronti integrati:

1. **Ragionamento automatico (Knowledge Representation):** sviluppo di un’ontologia OWL che formalizza la logica tattica (ruoli, resistenze, sinergie). Tramite il reasoner HermiT, il sistema deduce proprietà latenti che i dati numerici da soli non esplicitano.
2. **Apprendimento supervisionato (Machine Learning):** addestramento di modelli predittivi per la classificazione dei Tier Smogon, mettendo a confronto una baseline basata su statistiche grezze con un modello arricchito semanticamente.
3. **Risoluzione di vincoli (Teambuilder CSP):** utilizzando la conoscenza arricchita dall’ontologia (i ruoli inferiti) e le predizioni del Machine Learning (i Tier), viene implementato un risolutore CSP (Constraint Satisfaction Problem) capace di comporre una squadra di 6 Pokémon che rispetti vincoli di equilibrio tattico e varietà tipologica.

Elenco argomenti di interesse

Il progetto analizza l’integrazione di tre pilastri fondamentali:

- **Rappresentazione della conoscenza:** viene costruita una KB ontologica per trasformare dati grezzi in concetti tattici, fornendo al sistema una Background Knowledge (BK) che ne aumenta la capacità discriminante.
- **Apprendimento supervisionato:** vengono utilizzati modelli robusti (k-Nearest Neighbors e SVM) valutati tramite Nested Cross-Validation, con particolare attenzione alla gestione dello sbilanciamento delle classi e alla stabilità statistica.
- **Ricerca e risoluzione dei vincoli:** viene implementato un modulo di pianificazione che trasforma la conoscenza teorica in una soluzione pratica (squadra ottimale), garantendo il soddisfacimento di vincoli definiti dall’utente o dal dominio.

2. Rappresentazione della conoscenza e ragionamento

Sommario

Per superare i limiti dell’approccio puramente numerico, ho scelto di utilizzare un’ontologia per rappresentare la logica tattica del dominio. L’ontologia fornisce al sistema la Background Knowledge necessaria per comprendere le relazioni elementali e i ruoli strategici. In questo modo il sistema ragiona seguendo le regole del dominio.

Strumenti utilizzati

Il modulo è stato implementato in Python v. 3.14.3 integrando i seguenti strumenti:

- `owlready2`: è fondamentale per la manipolazione di ontologie OWL. Ho scelto questa libreria per le sue capacità di gestire la TBox (struttura) e ABox (individui) direttamente attraverso il codice, permettendo un’integrazione fluida con il dataset pandas.
- `Pandas`: è stato utilizzato per alimentare l’ontologia partendo dal dataset originale in formato CSV.
- `HermiT`: è stato scelto come motore di inferenza (reasoner) per la sua capacità di gestire la logica descrittiva complessa e le regole SWRL definite nella TBox.
- `sync_reasoner`: funzione specifica di owlready2 utilizzata per sincronizzare la base di conoscenza e calcolare i valori delle proprietà inferite.
- **SWRL (Semantic Web Rule Language):** ho utilizzato SWRL per definire regole di inferenza complesse che superano i limiti della logica descrittiva standard di OWL.

Decisioni di progetto

E’ stato scelto un dataset in formato CSV estratto da Kaggle, verificandone personalmente la correttezza dei dati al suo interno confrontandoli con i valori nel database di Smogon. Le ragioni della scelta risiedono nella ricchezza informativa delle feature presenti:

- *Name*: nome dell’esemplare di Pokémon;
- *Type 1*: tipo elementale primario del Pokémon;
- *Type 2*: tipo elementale secondario del Pokémon;
- *HP*: rappresenta la capacità di assorbimento danni;
- *Attack / Sp_attack*: definiscono il potenziale offensivo;
- *Defense / Sp_defense*: definiscono la resistenza ai colpi subiti;
- *Speed*: rappresenta l’ordine di azione durante il combattimento.

La scelta di mantenere la distinzione tra attacco fisico e speciale (analogamente per la difesa) permette di inferire specializzazioni offensive e difensive mirate. Ho deciso di non tenere in considerazione feature come *Name*, *Generation* e *Legendary,* i primi due perchè non rilevanti per il calcolo dei ruoli o della classificazione, l’ultimo per permettere al modello di valutare il potenziale di un Pokémon basandosi esclusivamente sulle sue capacità fisiche e tattiche.

Nonostante la disponibilità di vaste basi di conoscenza come DBPedia o Wikidata, ho scelto di sviluppare un’ontologia dedicata. Questa decisione è stata motivata dalla natura altamente specifica del dominio scelto: le basi di conoscenza infatti mancano delle definizioni tattiche necessarie a descrivere i ruoli di un Pokémon. Inoltre, lo sviluppo autonomo della TBox tramite Python e `owlready2` ha garantito il pieno controllo, assicurando che la conoscenza generata dal sistema fosse perfettamente integrata e

funzionale alle fasi successive di Machine Learning e CSP. Sebbene l’ontologia sia custom, essa è stata sviluppata seguendo lo standard di OWL 2, garantendo così la possibilità di futuri collegamenti con il Web Semantico.

Il primo passo nella costruzione dell’ontologia è stato la definizione delle classi, che sono state scelte secondo un criterio di pertinenza al dominio competitivo. L’attenzione è stata focalizzata sulle caratteristiche essenziali: l’esemplare di Pokémon, l’insieme di tipi elementali e i ruoli tattici che rivestono nel metagame in base alle statistiche base. Tutte le classi derivano dalla radice comune `owl:Thing` e sono state modellate come segue:

- ***Pokémon:*** rappresenta l’entità centrale del dominio. Ogni individuo di questa classe corrisponde ad un esemplare del dataset, fungendo da perno per tutte le asserzioni logiche.
- ***Type:*** classe terminologica che racchiude i 18 tipi elementali. Questa classe è fondamentale per definire le interazioni di forza e debolezza tramite relazioni tra individui.
- ***Role:*** rappresenta il target dell’inferenza logica. A differenza delle altre classi, questa non viene popolata manualmente, ma funge da “etichetta semantica” assegnata dal reasoner agli individui Pokémon che soddisfano determinati requisiti statistici.

Per rendere l’ontologia un sistema esperto, sono state definite relazioni specifiche per collegare i concetti:

- **Object Properties:** queste proprietà permettono di collegare esemplari di Pokémon ai loro attributi semantici e alle interazioni elementali:
  - *has_type* e *has_role*: sono le proprietà fondamentali di associazione. Mentre la prima è alimentata direttamente dal dataset, la seconda è il risultato del processo di ragionamento che lega il Pokémon alla sua funzione tattica inferita.
  - *is_weak_to* e *is_immune_to*: queste proprietà mettono in relazione individui della classe `Type`. Questa astrazione permette di modellare la “tabella dei tipi”.
  - **Property Chains:** sono state implementate due catene distinte per gestire la complessità difensiva:
    1. **Catena delle vulnerabilità:** $has_type \circ is_weak_to \sqsubseteq has_vulnerability$. Se il sistema sa che un Pokémon ha il tipo Terra e che la Terra è debole al tipo Erba, deduce istantaneamente la vulnerabilità del Pokémon al tipo Erba.
    2. **Catena delle immunità:** $has_type \circ is_immune_to \sqsubseteq has_immunity$. Il sistema identifica i tipi di danno che il Pokémon può ignorare completamente, ad esempio un Pokémon di tipo Volante che risulta immune alla Terra.

Questa scelta riduce drasticamente la ridondanza dei dati, inoltre senza le Property Chains avrei dovuto inserire manualmente migliaia di asserzioni; in questo modo il sistema le ricava logicamente in tempo reale. Inoltre, in fase di progettazione della TBox, ho scelto deliberatamente di omettere le resistenze di tipo. Questa decisione è stata presa per limitare la complessità logica dell’ontologia e ottimizzare i tempi di calcolo del ragionatore HermiT.

- **Data Properties:** sono state progettate per trasformare valori scalari in concetti logici utilizzabili dalle regole SWRL:
  - **FunctionalProperty:** ogni proprietà relativa alle statistiche (es. `has_high_attack`) è stata definita come funzionale. Ho intrapreso questa decisione per far sì che la Base di Conoscenza fosse coerente: un Pokémon non può avere contemporaneamente un valore alto e basso per la stessa statistica. Questo vincolo previene conflitti logici durante l’esecuzione del reasoner HermiT.
  - **Astrazione booleana:** ho applicato un’astrazione semantica per permettere al ragionatore semantico di operare su concetti di alto livello piuttosto che elaborare valori numerici grezzi, facilitando la classificazione automatica dei ruoli tattici. Ho scelto di convertire le statistiche continue, come ad esempio `Attack = 130`, in DataProperty booleane, come ad esempio `has_high_attack = true`, utilizzando le soglie competitive standard (es. 85 per la velocità, 90 per l'attacco) per definire cosa sia alto o basso nel metagame attuale.

Ho scelto di utilizzare il linguaggio SWRL per definire regole di inferenza capaci di combinare molteplici DataProperty booleane per dedurre il ruolo tattico di un Pokémon. Sono state implementate sette regole fondamentali che mappano i profili competitivi standard del dominio Pokémon:

- Definizione di “Sweeper” (Attaccanti veloci): il sistema identifica due varianti basate sulla natura dell’offesa (Fisica o Speciale). La regola logica impone che la presenza simultanea di alta velocità e alto potenziale d’attacco scateni l’assegnazione del ruolo:

```
Pokemon(?p), has_high_attack(?p, true), has_high_speed(?p, t
rue) -> has_role(?p, PhysicalSweeper),
Pokemon(?p), has_high_sp_attack(?p, true), has_high_speed(?
p, true) -> has_role(?p, SpecialSweeper)
```

- Definizione di “Wall” (Difensori): a differenza degli attaccanti, i difensori tattici vengono dedotti incrociando l’elevata capacità di assorbimento danni (HP) con le statistiche difensive:

```
 Pokemon(?p), has_high_hp(?p, true), has_high_defense(?p, tr
ue) -> has_role(?p, PhysicalWall),
```

```
 Pokemon(?p), has_high_hp(?p, true), has_high_sp_defense(?p,
true) -> has_role(?p, SpecialWall)
```

- Definizione di "Glass Cannon" (Fragilità offensiva): queste regole rappresentano l'aspetto più sofisticato dell'inferenza, poiché non valutano solo la presenza di punti di forza, ma anche la coesistenza di debolezze critiche (bassi HP e basse difese):

```
Pokemon(?p), has_high_attack(?p, true), has_low_defense(?p,
true), has_low_hp(?p, true) -> has_role(?p, PhysicalGlassCan
non),
Pokemon(?p), has_high_sp_attack(?p, true), has_low_defense(?
p, true), has_low_hp(?p, true) -> has_role(?p, SpecialGlassC
annon)
```

- Definizione di "Bulky Offense": regola dedicata a Pokémon lenti ma resistenti e potenti, definita tramite l'intersezione di alto attacco, alti HP e bassa velocità:

```
Pokemon(?p), has_high_attack(?p, true), has_high_hp(?p, tru
e), has_low_speed(?p, true) -> has_role(?p, BulkyOffense)
```

E' importante specificare che non a tutti gli esemplari presenti nel dataset viene assegnato un ruolo tattico poiché le regole SWRL sono basate su soglie statistiche rigide, perciò i Pokémon che presentano statistiche medie o bilanciate non soddisfano i requisiti necessari per l'inferenza di un ruolo specifico. Questi esemplari vengono classificati dal sistema come *None*. Questa è stata una scelta progettuale, in quanto l'obiettivo dell'ontologia è quello di evidenziare le eccellenze tattiche e le specializzazioni, permettendo così al modello di Machine Learning di migliorare la precisione della classificazione dei Tier, poiché i Pokémon privi di un ruolo definito tendono statisticamente a occupare le fasce competitive inferiori.

Per quanto riguarda la fase di ragionamento, la funzione `run_reasoning` carica la struttura definita precedentemente e avvia il processo di inferenza automatica. Una decisione fondamentale è stata l'attivazione del parametro `infer_property_values = True`, che permette a HermiT non solo di classificare gli individui nelle classi, ma di scrivere fisicamente i risultati delle Property Chains e delle regole SWRL all'interno dell'ontologia. L'obiettivo finale del ragionamento è la creazione di un dataset arricchito (OntoBK) da fornire al modulo di apprendimento supervisionato. Per fare ciò, è stata progettata una funzione `export_enriched_dataset` che estrae la conoscenza simbolica e la trasforma in feature numeriche e categoriche. Le decisioni chiave in questa fase includono:

- Risoluzione delle vulnerabilità reali: sfruttando la logica dell'ontologia, il sistema calcola il numero di debolezze effettive sottraendo le immunità inferite dalle

vulnerabilità grezze (**Vulnerabilità - Immunità = DebolezzeReali**). Per ragioni di semplicità e tempi di calcolo, questa metrica non considera le neutralizzazioni derivanti dal doppio tipo; è importante sottolineare che, nonostante tale scelta, la classificazione dei Tier ottenuta è risultata coerente con i dati di riferimento di Smogon, confermando che il modulo di Machine Learning è in grado di compensare tale approssimazione senza sbilanciare la solidità dei risultati finali.

- Classificazione dei Tier: ho implementato una funzione di classificazione euristica `calculate_smogon_tier` che assegna il Pokémon a un Tier competitivo incrociando tre fattori diversi:
  - Il Base Stat Total, ovvero la somma delle statistiche di base;
  - Solidità difensiva, ovvero il numero di debolezze reali dedotte dall’ontologia;
  - Specializzazione tattica, cioè la presenza o meno di un ruolo specifico inferito tramite SWRL.

Questa scelta progettuale permette di ottenere un target di classificazione coerente, infatti un Pokémon con statistiche medie ma con un ruolo tattico ben definito e poche debolezze può scalare i Tier, riflettendo ciò che accade nel metagame competitivo reale. E’ importante precisare che, a differenza del sistema ufficiale di Smogon che aggiorna i Tier mensilmente in base alla popolarità d’uso dei Pokémon, in questo progetto ho scelto di adottare una classificazione basata sul potenziale tecnico. Questa decisione è stata presa per garantire la stabilità della Base di Conoscenza: tale approccio assicura che il modulo di Machine Learning apprenda regole costanti e che il Teambuilder suggerisca squadre equilibrate basate sulla forza reale dei componenti.

Valutazione

Il primo criterio di valutazione è stato il superamento del test di coerenza tramite il reasoner HermiT. Il sistema ha confermato che la TBox (le regole e le classi) e la ABox (i dati dei Pokémon popolati dal CSV) non presentano conflitti logici. In particolare:

- Le Functional Properties hanno impedito con successo l’assegnazione di valori contraddittori (es. un Pokémon non è mai risultato contemporaneamente con alta e bassa velocità).
- Le Property Chains hanno popolato correttamente le relazioni di vulnerabilità e immunità per tutti gli esemplari senza errori di sovrapposizione.

L’efficacia del sistema è stata verificata analizzando la capacità di trasformare dati numerici in etichette tattiche. Il successo è evidenziato dalla generazione di 7 ruoli competitivi distinti che non erano presenti nel dataset originale.

Il successo finale di questa sezione risiede nella creazione del dataset OntoBK. La valutazione ha mostrato che il calcolo delle debolezze reali, seppur semplificato, ha fornito

una metrica di fragilità che ha permesso al ML di distinguere meglio tra Pokémon con statistiche simili ma Tier differenti. Inoltre, l'aggiunta delle feature `inferred_role` ha ridotto l'ambiguità del dataset, permettendo al modulo di classificazione di capire la funzione di un Pokémon prima ancora di valutarne la potenza. L'ontologia ha trasformato un database statico in una Base di Conoscenza dinamica, gettando le basi per l'incremento di accuratezza osservato nel modulo di ML e per la precisione del Teambuilder CSP.

3. Apprendimento supervisionato

Sommario

L'obiettivo di questo modulo è la progettazione e la validazione di un sistema di classificazione capace di predire il Tier competitivo di un Pokémon. Il task è stato approcciato come un problema di apprendimento supervisionato multiclasse, dove il modello deve assegnare ogni esemplare a una delle cinque categorie principali: Uber, OverUsed (OU), UnderUsed (UU), RarelyUsed (RU) e NeverUsed (NU).

La sperimentazione è stata strutturata su due livelli comparativi:

1. **Modello Baseline:** addestrato esclusivamente sulle statistiche base per testare i limiti della pura analisi numerica.
2. **Modello Enriched (OntoBK):** addestrato integrando le feature semantiche dedotte dall'ontologia.

Strumenti utilizzati

Il progetto è stato implementato utilizzando le seguenti librerie specializzate:

- `Pandas` e `Numpy`: per la manipolazione del dataset in formato CSV e la gestione degli array multidimensionali.
- `Matplotlib`: per la creazione dei grafici di valutazione e delle curve di apprendimento.
- `Scikit-learn`: framework centrale che ha fornito gli algoritmi di classificazione e gli strumenti di validazione:
  - `StandardScaler`: per la normalizzazione delle feature numeriche.
  - `SVC`, `KNeighborsClassifier`: algoritmi scelti per l'addestramento.
  - `GridSearchCV` e `cross_validate`: per l'ottimizzazione degli iperparametri e la validazione robusta.
  - `StratifiedKFold`: per garantire una suddivisione dei dati che rispetti la proporzione delle classi target.

- `learning_curve`: per la diagnostica della convergenza e dell'andamento dell'apprendimento.
- `make_scorer, accuracy_score, precision_score, recall_score, f1_score` : per la valutazione quantitativa e la misurazione delle performance dei classificatori.

Decisioni di progetto

L'apprendimento supervisionato è una branca dell'apprendimento automatico in cui l'agente viene addestrato su un insieme di dati con etichette. L'obiettivo principale è far sì che l'algoritmo impari una relazione tra gli input e gli output in modo da poter fare previsioni accurate. A seconda della natura delle feature target, l'apprendimento supervisionato si articola in due compiti:

1. Regressione: quando la variabile target è continua.
2. Classificazione: quando la variabile target appartiene a un dominio di valori discreto.

Per questo progetto, l'obiettivo è predire il Tier competitivo di un Pokémon, una categoria discreta che ne definisce il posizionamento nel metagame ufficiale. Pertanto, è stato adottato un approccio di classificazione multiclasse. Ho selezionato come target la feature `smogon_tier`, escludendo variabili che potrebbero introdurre fenomeni di target leakage per garantire che il modello apprenda solo dalle caratteristiche tecniche e semantiche degli esemplari.

La fase di progettazione è stata guidata dalla necessità di superare i limiti dei modelli puramente statistici. La decisione cardine di questa fase è stata l'inclusione delle feature derivate dall'ontologia all'interno del set di addestramento, con l'obiettivo di fornire al sistema una sensibilità tattica simile a quella di un giocatore esperto. Per implementare questa strategia, ho realizzato la funzione `load_datasets`, incaricata di trasformare i dati grezzi in conoscenza strutturata per gli algoritmi tramite i seguenti passaggi:

- Viene eseguito il caricamento del dataset tramite la libreria `pandas` e la contestuale rimozione delle istanze prive di etichetta target, garantendo che l'agente non venga addestrato su esempi rumorosi o incompleti;
- La funzione genera due diverse matrici di feature per validare l'apporto semantico:
  - Baseline (`X_base`): contenente esclusivamente le sei statistiche numeriche fondamentali.
  - OntoBK (`X_enriched`): il set arricchito che integra la feature `real_weaknesses` e i ruoli tattici inferiti dal ragionamento logico.
- Trasformazione semantica: poiché modelli matematici come la SVM e il k-NN richiedono input numerici per operare nello spazio geometrico, è stato necessario gestire la variabile categorica `inferred_role`. Ho optato per una trasformazione in variabili binarie tramite la funzione `pd.get_dummies`: questa operazione scompone ogni

ruolo in colonne indipendenti contenenti valori 0 o 1. Tale scelta permette al classificatore di pesare correttamente l’importanza di ogni specifica funzione tattica senza introdurre una gerarchia artificiale o un ordinamento arbitrario tra i ruoli.

Ho selezionato e confrontato due modelli basati su paradigmi differenti per testare la solidità della conoscenza ontologica:

- **k-Nearest Neighbors (k-NN)**: è un classificatore basato sulla similarità. Classifica ogni nuovo Pokémon confrontandolo con i k esemplari più vicini nello spazio delle feature. E’ stato scelto per verificare se la conoscenza dei ruoli e delle debolezze aiuti a raggruppare i Pokémon in vicinati più coerenti.
- **SVM (Support Vector Machine)**: è un modello in grado di mappare i dati in uno spazio a più dimensioni per trovare confini non lineari, scelto per gestire le relazioni non lineari tra le statistiche base e il successo competitivo.

Gli iperparametri sono parametri del modello che non vengono appresi durante l’addestramento, ma devono essere fissati preventivamente, influenzando drasticamente le prestazioni e la complessità. Per la loro ricerca ho utilizzato la tecnica della GridSearch con Cross Validation. Questa strategia esplora in modo esaustivo una griglia di valori predefiniti alla ricerca della combinazione ottimale.

Per il k-NN:

```
"k-NN (Instance-based)": (KNeighborsClassifier(), {
            'classifier__n_neighbors': [3, 5, 11],
            'classifier__weights': ['uniform', 'distance']
        })
```

- `classifier__n_neighbors`: rappresenta il numero di vicini da considerare. Ho testato i valori [3, 5, 11] per bilanciare il dettaglio locale e la stabilità statistica.
- `classifier__weights`: definisce l’influenza dei vicini. Ho confrontato *uniform* e *distance* per valutare se pesare maggiormente i Pokémon geometricamente più simili portasse a risultati migliori.

Per la SVM:

```
"SVM (Kernel RBF)": (SVC(random_state=42), {
            'classifier__C': [0.1, 1, 10],
            'classifier__kernel': ['rbf']
        })
```

- `classifier_C`: è il parametro di regolarizzazione. Ho testato [0.1, 1, 10]. Un valore basso predilige un margine più ampio, mentre un valore alto spinge il modello a

classificare correttamente ogni singolo punto del training set.

- `classifier_kernel`: impostato su *rbf* per permettere confini di decisione curvi, necessari per separare classi con statistiche parzialmente sovrapponibili.

Per ottenere una stima realistica e non ottimistica, è stata utilizzata una Nested Cross-Validation. Questa tecnica separa nettamente la fase di addestramento e ottimizzazione dalla fase di test finale.

- Ciclo esterno: suddivide l’intero dataset in 5 fold tramite `cross_validate`. Ad ogni iterazione, un fold viene utilizzato come test set finale e gli altri da training.
- Ciclo interno: all’interno di ogni fold di training viene eseguita una `GridSearchV` a 3 fold per identificare la configurazione ottimale degli iperparametri, garantendo che questi non vengano scelti sbirciando i dati di test finale.

Alla fine del processo si ottiene una media dei punteggi ottenuti sui 5 set di test del ciclo esterno, modalità che fornisce una stima realistica.
Inoltre, tramite la libreria `matplotlib`, ho generato i grafici raffiguranti le curve di apprendimento e il confronto dell’efficacia ontologica tra SVM e k-NN.

Valutazione

La valutazione delle prestazioni si è basata sul confronto diretto tra i modelli addestrati sulle sole statistiche numeriche e quelli integrati con la conoscenza ontologica. Per garantire una visione d’insieme corretta, sono state analizzate quattro metriche fondamentali: Accuracy, Precision, Recall e F1-Score. Data la natura del dominio, i valori di Precision, Recall e F1 sono stati calcolati con media Macro, pesando equamente ogni Tier indipendentemente dalla sua numerosità nel dataset.

Risultati della fase baseline

In questa fase, i modelli hanno dimostrato una discreta capacità predittiva basata esclusivamente sui dati numerici, ma con una marcata instabilità.

Nel caso del k-NN:

```
Statistiche Finali
   Accuracy:  Media = 86.85% | Std = 3.15% | Var = 9.9024
   Precision: Media = 79.67% | Std = 6.74% | Var = 45.4049
   Recall:    Media = 74.86% | Std = 5.08% | Var = 25.8379
   F1-Score:  Media = 76.31% | Std = 5.38% | Var = 28.9218
```

il valore dell’Accuracy è apparentemente alto, ma ingannevole se non confrontato con le altre metriche: infatti il valore dell’F1-Score è sensibilmente più basso dell’accuratezza, indicando difficoltà nel gestire le classi meno frequenti. Inoltre, una deviazione standard del

5.38% sull'F1-Score suggerisce che il modello è molto sensibile alla suddivisione dei dati, conclusione che dipende anche dall'alta varianza, con un valore di 28.92.

```
Addestramento Baseline (Senza Ontologia)...
 Valutazione di k-NN (Instance-based)...
 Migliori Iperparametri trovati per k-NN (Instance-based)
    Fold 1: {'n_neighbors': 5, 'weights': 'distance'}
    Fold 2: {'n_neighbors': 5, 'weights': 'distance'}
    Fold 3: {'n_neighbors': 5, 'weights': 'distance'}
    Fold 4: {'n_neighbors': 11, 'weights': 'distance'}
    Fold 5: {'n_neighbors': 5, 'weights': 'distance'}
```

L'analisi degli iperparametri scelti per il modello k-NN durante questa fase fornisce prove concrete sulla natura del dataset e conferma le difficoltà di generalizzazione evidenziate in precedenza: la variazione di n_neighbors nel Fold 4 è il segnale tecnico dell'instabilità precedentemente rilevata, e ciò dimostra che, senza il supporto dell'ontologia, il modello è indeciso sulla struttura dei dati. La scelta costante di `weights: 'distance'` suggerisce che nel dominio del teambuilding esistono gruppi di Pokémon con statistiche simili che tendono a ricoprire ruoli simili. Il fatto che il modello debba oscillare tra k = 5 e k = 11 per trovare l'accuratezza ottimale conferma che le sole feature numeriche sono insufficienti.

L'analisi delle statistiche finali per la SVM rivela un modello più performante del k-NN, ma ancora limitato dalla mancanza di una struttura di conoscenza profonda:

```
Statistiche Finali
   Accuracy:  Media = 88.90% | Std = 2.44% | Var = 5.9699
   Precision: Media = 82.06% | Std = 6.39% | Var = 40.8097
   Recall:    Media = 77.56% | Std = 3.86% | Var = 14.8706
   F1-Score:  Media = 78.41% | Std = 4.13% | Var = 17.0591
```

valutando il valore dell'accuratezza, notiamo come il modello raggiunge una capacità di classificazione corretta superiore rispetto al k-NN. Il valore della Precision è solido, ma accompagnato da una deviazione standard elevata (6.39%); questo indica che la SVM soffre di fluttuazioni marcate a seconda del subset di addestramento. Il Recall rappresenta il punto debole relativo al modello, infatti la SVM fatica a identificare tutti i casi positivi lasciando molti falsi negativi sul campo. La sintesi tra precisione e richiamo conferma un salto di qualità rispetto al 76.31% del k-NN, posizionando la SVM come una base di partenza migliore per l'integrazione ontologica.

Gli iperparametri selezionati per tutti i 5 fold sono i seguenti:

```
Valutazione di SVM (Kernel RBF)...
Migliori Iperparametri trovati per SVM (Kernel RBF)
   Fold 1: {'C': 10, 'kernel': 'rbf'}
   Fold 2: {'C': 10, 'kernel': 'rbf'}
   Fold 3: {'C': 10, 'kernel': 'rbf'}
   Fold 4: {'C': 10, 'kernel': 'rbf'}
   Fold 5: {'C': 10, 'kernel': 'rbf'}
```

l’utilizzo del kernel rbf conferma che la classificazione delle istanze richiede una separazione non lineare. Il modello proietta i dati in uno spazio iperdimensionale per trovare confini che nel dataset originale sono sovrapposti. Il parametro di regolarizzazione impostato a 10 indica una scelta “aggressiva”: un valore di C più alto spinge il modello a classificare correttamente ogni punto del training set, accettando un margine di separazione più stretto.

Risultati della fase OntoBK

Nonostante la natura semplice del k-NN, l’apporto semantico ha prodotto un miglioramento evidente:

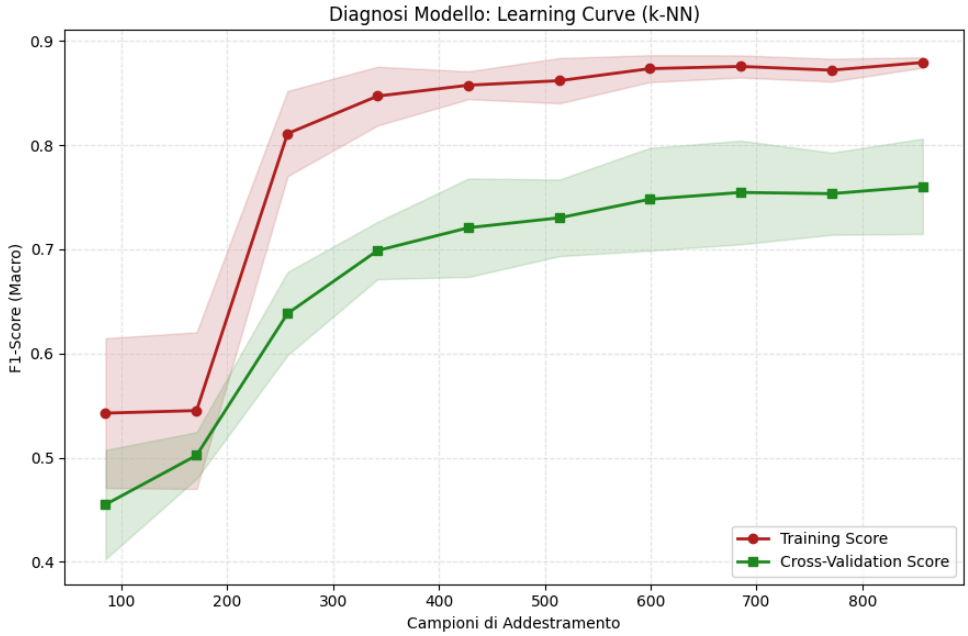
```
Statistiche Finali
   Accuracy:  Media = 86.38% | Std = 1.98% | Var = 3.9250
   Precision: Media = 82.73% | Std = 2.48% | Var = 6.1408
   Recall:    Media = 79.13% | Std = 2.65% | Var = 6.9965
   F1-Score:  Media = 80.32% | Std = 2.65% | Var = 7.0459
```

il valore dell’Accuracy rimane pressoché stabile rispetto alla baseline (86.85%), indicando che il numero delle istanze correttamente classificate non è cambiato drasticamente. Si registra un incremento significativo della Precision rispetto al 79.67% iniziale. La metrica che ne beneficia di più è il Recall, salendo di circa 5 punti percentuali. L’ontologia aiuta l’algoritmo a non tralasciare istanze che prima erano statisticamente ambigue. L’F1-Score riflette questo nuovo equilibrio, superando la soglia dell’80%.

L’analisi degli iperparametri per il k-NN fornisce la prova definitiva di come l’ontologia abbia semplificato il panorama dei dati:

```
Addestramento OntoBK (Con Ontologia)...
 Valutazione di k-NN (Instance-based)...
 Migliori Iperparametri trovati per k-NN (Instance-based)
    Fold 1: {'n_neighbors': 5, 'weights': 'distance'}
    Fold 2: {'n_neighbors': 3, 'weights': 'distance'}
    Fold 3: {'n_neighbors': 5, 'weights': 'distance'}
    Fold 4: {'n_neighbors': 3, 'weights': 'distance'}
    Fold 5: {'n_neighbors': 3, 'weights': 'distance'}
```

si nota una tendenza verso valori di k più bassi rispetto alla fase precedente, in particolare in tre fold su cinque, il modello ha preferito k = 3; ciò accade perché l’ontologia ha separato nettamente le classi. Il parametro `weights: ‘distance’` viene confermato come parametro ottimale per tutti i fold.



Analizzando la curva di apprendimento, si osserva che il Training Score si stabilizza rapidamente intorno a 0.88, mentre il punteggio di Cross-Validation fatica a superare la soglia di 0.76. Nonostante il contributo semantico dell’ontologia, la persistenza di un divario netto tra le due curve e l’ampia banda di varianza indicano che il modello continua a risentire di un fenomeno di overfitting statistico, segno che la conoscenza appresa dai dati di addestramento non è ancora pienamente trasferibile.

L’analisi delle statistiche finali per la SVM con l’integrazione OntoBK conferma come l’apporto semantico abbia trasformato il classificatore in un sistema ad alte prestazioni ed estremamente stabile:

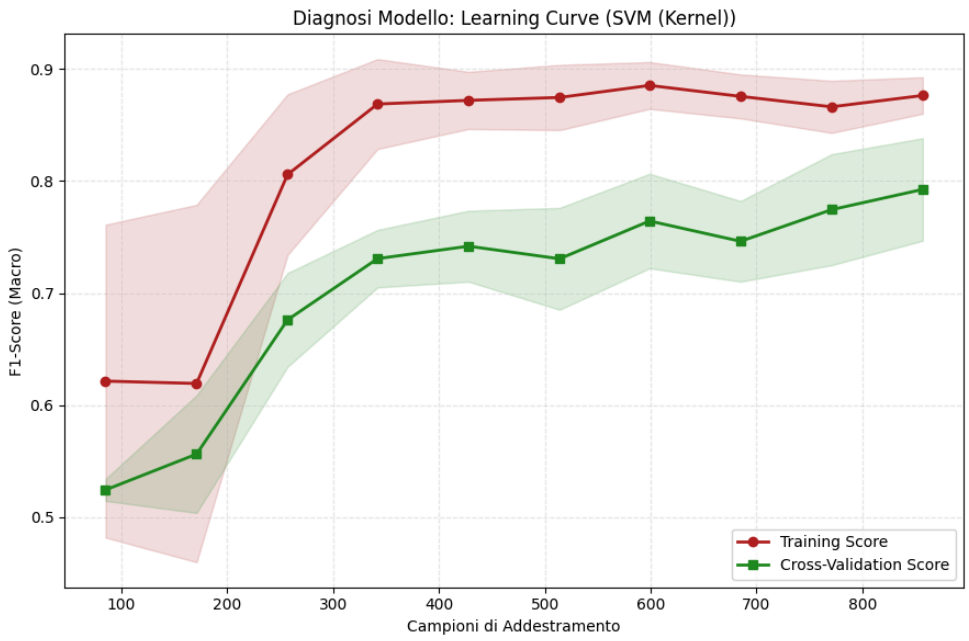
```
Statistiche Finali
   Accuracy:  Media = 92.35% | Std = 0.70% | Var = 0.4914
   Precision: Media = 88.80% | Std = 2.23% | Var = 4.9868
   Recall:    Media = 86.64% | Std = 2.29% | Var = 5.2573
   F1-Score:  Media = 87.46% | Std = 2.31% | Var = 5.3195
```

si registra un balzo significativo rispetto all’88.90% della baseline. Il modello ora classifica correttamente la quasi totalità delle istanze fornite. La capacità del modello di evitare falsi positivi è aumentata di circa 6.7 punti percentuali, perciò ogni assegnazione a una classe è molto più affidabile. Il dato più impattante è il Recall, con un incremento di oltre 9 punti percentuali (da 77.56% a 86.64%). L’ontologia ha permesso al modello di identificare correttamente istanze che prima venivano confuse tra classi. L’F1-Score conferma un equilibrio ottimale tra precisione e richiamo grazie alla nuova profondità informativa. Il successo dell’integrazione ML + OntoBK non si misura solo nei picchi di punteggio, ma anche nella riduzione dell’incertezza: la deviazione standard dell’accuratezza è scesa dal 2.44% allo 0.70%. Inoltre, con l’OntoBK, la varianza della precisione crolla a 4.98, dimostrando che la conoscenza semantica ha rimosso il rumore statistico.

L’invarianza degli iperparametri della SVM con OntoBK porta a una conclusione sulla natura del progetto:

```
Valutazione di SVM (Kernel RBF)...
Migliori Iperparametri trovati per SVM (Kernel RBF)
   Fold 1: {'C': 10, 'kernel': 'rbf'}
   Fold 2: {'C': 10, 'kernel': 'rbf'}
   Fold 3: {'C': 10, 'kernel': 'rbf'}
   Fold 4: {'C': 10, 'kernel': 'rbf'}
   Fold 5: {'C': 10, 'kernel': 'rbf'}
```

anche se il valore di C è rimasto costante, il modello ha ottenuto una Recall molto superiore (86.64% contro 77.56%). Questo indica che l’arricchimento ontologico ha reso le classi meno ambigue.



L’analisi della curva di apprendimento rivela una dinamica di apprendimento molto più bilanciata rispetto al k-NN: le curve di training e di test mostrano un chiaro trend di crescita e una progressiva convergenza verso il finale del grafico. Questo andamento conferma visivamente che la SVM è l’algoritmo che ha saputo integrare con maggior successo la Background Knowledge semantica, riducendo l’instabilità del modello e dimostrando una robusta capacità di generalizzazione.

4. Risoluzione di vincoli

Sommario

Il modulo di Teambuilding rappresenta il motore decisionale del sistema esperto, dove la conoscenza estratta dall’ontologia e validata dal Machine Learning viene trasformata in una decisione pratica. Il problema della creazione di una squadra bilanciata e competitiva è stato modellato come un Problema di Soddisfacimento di Vincoli (CSP). L’obiettivo è identificare una combinazione di 6 Pokémon che massimizzi l’efficacia del team rispettando rigidi vincoli di bilanciamento e copertura.

Strumenti utilizzati

La modellazione dei vincoli è stata affidata a:

- `python-constraint`: libreria specializzata nella risoluzione di problemi di soddisfacimento di vincoli su domini finiti. Supporta diversi algoritmi di ricerca e ha permesso di implementare vincoli personalizzati.

Decisioni di progetto

La sfida principale nella progettazione del Teambuilder è stata la traduzione della competitività in vincoli matematici precisi. Ho scelto di utilizzare un approccio CSP invece di semplici clausole logiche per gestire meglio l’aspetto combinatorio del problema, definendo una tripla (X, D, C):

- Variabili (X): un insieme di 6 variabili discrete $\{S_1, \ldots, S_6\}$ rappresentanti gli slot del team;
- Dominio (D): per gestire l’esplosione combinatoria, ho implementato un’euristica di pre-filtraggio. Invece di esplorare l’intero spazio campionario, il sistema opera su un pool dinamico di 80 elementi dove una funzione di pre-validazione scarta il pool se non contiene i requisiti minimi (almeno un attaccante, un difensore e 6 tipi diversi). Questa tecnica riduce drasticamente lo spazio degli stati senza compromettere la qualità della soluzione;
- Vincoli (C): ho definito i seguenti vincoli fondamentali:
  - Vincolo di bilanciamento (Tier): limita la potenza complessiva per evitare team composti solo da leggendari. La regola permette al massimo 1 Pokémon della classe Uber e 2 della classe OverUsed, forzando il sistema a cercare sinergie anche tra Pokémon meno potenti;
  - Vincolo di diversità elementale: per evitare che il team sia vulnerabile a un singolo tipo avversario, il risolutore impone che i tipi primari dei 6 membri siano tutti differenti tra loro;
  - Vincolo di sinergia tattica: sfruttando la Background Knowledge, il sistema impone la presenza di almeno un difensore (Wall) e almeno due attaccanti (Sweeper), assicurando che la squadra sia un’unità funzionale;
  - Vincolo di unicità: impedisce l’inserimento di duplicati dello stesso Pokémon, garantendo la varietà richiesta dal regolamento standard.

Il risolutore utilizza un processo di Backtracking Search con Pruning dell’albero delle decisioni. Se un Pokémon viola un vincolo di tipo o di Tier, l’algoritmo interrompe immediatamente l’esplorazione di quel ramo, ottimizzando i tempi di risposta.

Valutazione

L’analisi del modulo CSP evidenzia il successo dell’integrazione tra logica e statistica. Vi è coerenza con la conoscenza di dominio, infatti a differenza di un selettore puramente casuale o statistico, il Teambuilder produce squadre tatticamente sensate. Il vincolo di sinergia garantisce che il team abbia sempre una struttura bilanciata. Inoltre, il Teambuilder funge da validazione finale per l’ontologia: infatti, senza i ruoli inferiti dal ragionatore HermiT, il sistema non avrebbe i criteri necessari per bilanciare la squadra, dimostrando

che la Background Knowledge è il vero motore decisionale del sistema. Infine, grazie al pre-filtraggio del pool a 80 elementi, il tempo di ricerca della soluzione è pressoché istantaneo.

```
FASE 4: Risoluzione Vincoli (Teambuilder CSP)
Ricerca della prima squadra ottimale nell'albero delle combinazioni...

 --- SQUADRA PERFETTA TROVATA! ---
 > Slot_1: Farfetch'd          | Tipo: Normal   | Tier: RarelyUsed | Ruolo: PhysicalGlassCannon
 > Slot_2: Aerodactyl          | Tipo: Rock     | Tier: UnderUsed  | Ruolo: PhysicalSweeper
 > Slot_3: Manectric           | Tipo: Electric | Tier: UnderUsed  | Ruolo: SpecialSweeper
 > Slot_4: Mega Slowbro        | Tipo: Water    | Tier: UnderUsed  | Ruolo: PhysicalWall
 > Slot_5: Celebi              | Tipo: Psychic  | Tier: Uber       | Ruolo: SpecialWall
 > Slot_6: Tangrowth           | Tipo: Grass    | Tier: UnderUsed  | Ruolo: BulkyOffense
```

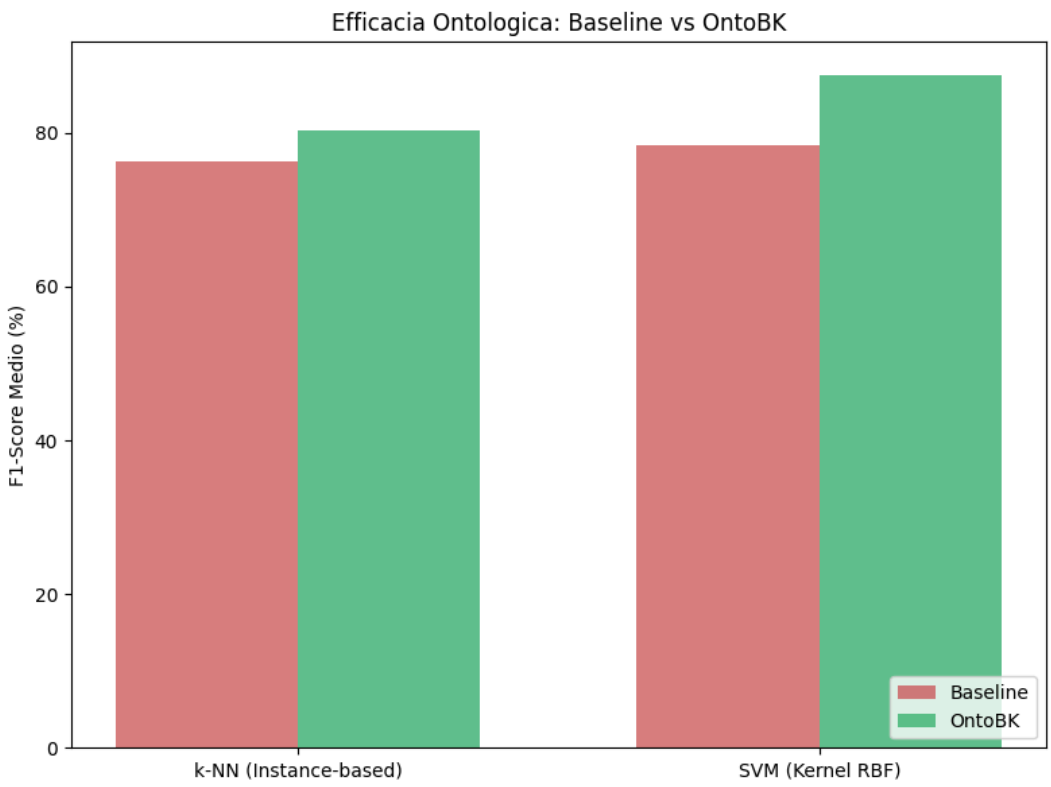
Si osserva che ogni Pokémon selezionato appartiene a un tipo primario differente, confermando che il vincolo di copertura ha operato correttamente, inoltre il team generato include sia profili offensivi (Sweeper) che difensivi (Wall). Il team rispetta anche il vincolo di potenza imposto: è presente esattamente un Pokémon di tier Uber e nessun Pokémon di tier OverUsed, con una forte base UnderUsed. Questo dimostra che il sistema non si limita a scegliere i più forti, ma cerca la migliore combinazione logica possibile entro i limiti di budget di potenza definiti.

Conclusioni

L’analisi comparativa permette di trarre una conclusione: l’integrazione tra Ragionamento Logico e Apprendimento Automatico è stata la chiave del successo del progetto. Mentre le sole statistiche numeriche fornivano una base di partenza incerta e soggetta a errori grossolani sui casi limite, l’OntoBK ha agito come un regolarizzatore semantico. L’arricchimento dei dati con i ruoli tattici e le debolezze reali ha permesso ai modelli di:

1. Risolvere le ambiguità: Pokémon con statistiche simili ma Tier diversi sono ora classificati correttamente grazie al contesto strategico;
2. Massimizzare la stabilità: la riduzione della varianza prova che il sistema ha acquisito una conoscenza solida e non basata sul caso;
3. Supportare il Teambuilder: senza l’accuratezza del 92.35% raggiunta, la fase di risoluzione dei vincoli non avrebbe potuto garantire la creazione di squadre bilanciate e competitive, rischiando di basare la logica su Tier errati.

Il successo di questa integrazione è reso evidente dal confronto diretto tra le prestazioni dei modelli prima e dopo l’intervento dell’ontologia:



Come si evince dal grafico, l’introduzione della conoscenza di dominio ha prodotto un incremento netto dell’F1-Score per entrambi gli algoritmi. In particolare, il modello SVM ha mostrato la risposta più significativa all’arricchimento semantico, superando la soglia dell’87% e dimostrando una stabilità (Std = 0.70%) che la sola base numerica non poteva garantire.

Sviluppi futuri

Nonostante l’efficacia dimostrata dall’attuale architettura ibrida, il sistema presenta ampi margini di espansione per aumentare la granularità del ragionamento e l’accuratezza delle decisioni: un’evoluzione dell’ontologia prevede l’integrazione delle Abilità, ovvero delle caratteristiche speciali che influenzano le prestazioni in lotta, e degli strumenti tenuti, ovvero oggetti che conferiscono bonus o effetti speciali. Questo permetterebbe al ragionatore HermiT di inferire ruoli ancora più specifici, spostando la logica del sistema da una base prettamente statistica a una puramente strategica.

Inoltre, il sistema esperto potrebbe essere integrato con un agente addestrato tramite Reinforcement Learning per simulare battaglie in tempo reale. In questo scenario, l’ontologia fornirebbe la conoscenza di base, il CSP comporrebbe il team e l’agente RL verificherebbe l’efficacia della squadra, creando un ciclo di feedback continuo per il raffinamento dei vincoli tattici.

Riferimenti bibliografici

[1] Tiering FAQ - Smogon University. https://www.smogon.com/ingame/battle/tiering-faq

[2] Pokemon | SV | Smogon Strategy Pokedex. https://www.smogon.com/dex/sv/pokemon/

[3] Pandas: Getting Started. https://pandas.pydata.org/docs/getting_started/index.html#getting-started

[4] Pandas: User Guide. https://pandas.pydata.org/docs/user_guide/index.html

[5] Scikit-learn: Nested versus non-nested cross-validation. https://scikit-learn.org/stable/auto_examples/model_selection/plot_nested_cross_validation_iris.html

[6] Matplotlib: Quick Start. https://matplotlib.org/stable/users/explain/quick_start.html

[7] Scikit-learn: K-NeighborsClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html#sklearn.n

[8] Scikit-learn: SVMClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html

[9] python-constraint. https://pypi.org/project/python-constraint/

[10] Supervised Machine Learning: D. Poole, A. Mackworth: Artificial Intelligence: Foundation of Computational Agents. [Ch.7]

[11] Knowledge Graph and Ontologies: D. Poole, A. Mackworth: Artificial Intelligence: Foundation of Computational Agents. [Ch.16]

[12] Reasoning with Constraints: D. Poole, A. Mackworth: Artificial Intelligence: Foundation of Computational Agents. [Ch.4]